

技術面策略報告整理

本文件依照 `finalProject.pdf` 的報告要求，整理 `technical_strategy` 專案可以在簡報中說明的內容。

1. Goal / Input

研究目標

`technical_strategy` 的目標是：

使用台積電歷史股價與技術指標，預測下一個交易日股價是否上漲，並檢驗這個預測訊號是否能形成有效的交易策略。

此專案屬於監督式學習中的二元分類問題。

模型預測目標為：

`target = 1`：下一個交易日收盤價高於今天收盤價

`target = 0`：下一個交易日收盤價未高於今天收盤價

資料來源

資料來源為 Yahoo Finance，程式透過 `yfinance` 下載每日 OHLCV 股價資料。

預設股票代號：

`2330.TW`

原始資料欄位包含：

`Open`

`High`

`Low`

Close

Volume

輸入格式

此專案預設不需要手動上傳 CSV，而是由使用者在 Streamlit 介面輸入：

- 股票代號
- 開始日期
- 結束日期
- validation 比例
- test 比例
- threshold 設定
- 交易成本假設

系統會自動下載股價資料，並轉換成模型訓練所需的技術指標特徵。

資料前處理

前處理主要在 `technical_system.py` 與 `technical_app.py` 中完成。

主要技術指標特徵包含：

特徵	說明
<code>return_1d</code>	單日報酬率
<code>ma_5</code>	5 日移動平均
<code>ma_20</code>	20 日移動平均
<code>ma_ratio</code>	<code>ma_5 / ma_20</code> ，衡量短期均線相對中期均線的位置
<code>bias_5</code>	收盤價相對 5 日均線的乖離率
<code>bias_20</code>	收盤價相對 20 日均線的乖離率
<code>vol_chg</code>	成交量變化率
<code>rsi_14</code>	14 日 RSI
<code>macd</code>	百分比 MACD
<code>macd_signal</code>	MACD 訊號線
<code>macd_hist</code>	MACD 柱狀圖

缺失值處理

缺失值主要來自：

- 移動平均、RSI 等 rolling window 指標
- 報酬率與成交量變化率
- 使用明日收盤價建立 target 時產生的最後一筆缺失值

程式處理方式為：

```
out = out.replace([np.inf, -np.inf], np.nan)
out = out.dropna().copy()
```

也就是將無限值轉為缺失值，並刪除仍有缺失值的資料列。

數值縮放

並非所有模型都需要縮放，因此專案依模型特性處理：

- `LogisticRegression_Ridge`：使用 `StandardScaler`
- `PCA_LogReg`：先使用 `StandardScaler`，再做 PCA 降維
- `RandomForest`：不需要標準化
- `XGBoost`：若環境有安裝，通常也不需要標準化

線性模型使用 `scikit-learn` `Pipeline` 將 `scaling` 與模型訓練串接，避免資料處理流程混亂。

2. Modeling

使用的方法

本專案使用監督式二元分類模型，輸入為技術指標特徵，輸出為下一個交易日上漲的機率：

`P(up)`

候選模型包含：

模型	說明
<code>LogisticRegression_Ridge</code>	使用 L2 regularization 的線性分類模型
<code>PCA_LogReg</code>	StandardScaler + PCA 降維 + Logistic Regression
<code>RandomForest</code>	可學習非線性關係的樹模型

模型	說明
XGBoost	若環境有安裝，則加入梯度提升模型比較

Null Model / 比較基準

本專案的主要比較基準是 Buy & Hold。

Buy & Hold 指的是：

在測試期間一開始買入股票，並一路持有到測試期間結束。

使用 Buy & Hold 作為基準，是因為如果主動交易策略無法明顯優於單純持有，則代表模型訊號在實務交易上不一定有價值。

比較時會觀察：

- 策略總報酬是否高於 Buy & Hold
- 最大回撤是否低於 Buy & Hold
- Sharpe ratio 是否較好
- 權益曲線是否更穩定

模型訓練方式

資料依時間順序切分為：

Train / Validation / Test

預設比例為：

Train：60%

Validation：20%

Test：20%

這裡不使用隨機切分，原因是股票資料屬於時間序列資料。如果隨機打散，可能會讓未來資訊混入訓練資料，造成資料洩漏。

訓練流程：

1. 每個模型只使用 training split 訓練。
2. 使用 validation split 比較模型與搜尋 threshold。
3. 使用 test split 作為最後樣本外績效報告。

Threshold Search

模型輸出的是下一日上漲機率：

P(up)

交易規則會將機率轉成部位：

target_position = 1 if P(up) ≥ threshold
target_position = 0 otherwise

Streamlit app 可在 validation set 上自動搜尋 threshold，選出 validation 策略報酬較好的 threshold，再套用到 test set。

Hyperparameters

目前模型使用固定候選參數，而不是完整 grid search。

模型	主要參數
Logistic Regression	penalty='l2' , C=1.0 , max_iter=2000
PCA Logistic Regression	PCA(n_components=0.90) , C=0.5
Random Forest	n_estimators=500 , max_depth=6 , min_samples_leaf=10
XGBoost	n_estimators=500 , max_depth=4 , learning_rate=0.01 , subsample=0.8 , colsample_bytree=0.8

3. Results

分類績效指標

專案會顯示以下分類指標：

- Accuracy
- Precision
- Recall
- F1 score

這些指標用來回答：

模型是否能正確預測下一個交易日漲跌方向？

交易績效指標

因為此專案最終目標是交易策略分析，所以不能只看分類準確率。

app 也會計算投資績效：

- Strategy total return
- Buy & Hold total return
- Strategy maximum drawdown
- Strategy Sharpe ratio
- Entry count
- Holding ratio
- Total trading cost

主要評估指標選擇

目前專案主要使用 validation strategy total return 選出最佳模型。

原因是：

金融資料中，*accuracy* 高不代表策略一定賺錢。

例如模型可能略高於隨機猜測，但如果交易成本過高、錯誤發生在大跌時、或回撤太大，實務上仍然不是好策略。

因此本專案同時觀察：

- 預測能力：Accuracy、Precision、Recall、F1
- 投資能力：Return、Max Drawdown、Sharpe、交易次數、持有比例

指標是否適合此資料

股票漲跌資料具有高雜訊、非穩定分布、市場 regime 變化等特性，因此單一分類指標不足以評估策略好壞。

使用交易績效指標的原因：

- `strategy_total_return` 可觀察策略是否有獲利能力
- `buy_hold_total_return` 提供被動持有基準
- `max_drawdown` 可衡量最大風險
- `Sharpe ratio` 可衡量風險調整後報酬
- `entry_count` 可檢查策略是否過度交易或幾乎不交易

- `holding_ratio` 可觀察策略曝險程度

改善是否顯著

目前 app 會提供 validation 與 test split 的樣本外績效，但尚未加入正式統計顯著性檢定。

目前可以用以下方式初步判斷改善是否有意義：

- test strategy return 是否高於 Buy & Hold
- max drawdown 是否低於 Buy & Hold
- Sharpe ratio 是否改善
- 交易次數是否合理
- holding ratio 是否符合策略邏輯
- test set 表現是否與 validation set 方向一致

未來可加入更嚴謹的方法：

- Walk-forward validation
- Bootstrap confidence interval
- Deflated Sharpe Ratio
- 多模型選擇偏誤修正
- 與隨機訊號或均線策略等更強 baseline 比較

4. Demo

執行方式

在專案根目錄執行：

```
streamlit run technical_strategy/technical_app.py
```

Demo 建議流程

1. 設定股票代號與日期區間。
2. 設定 validation / test 比例。
3. 設定 threshold 或開啟 auto-search threshold。
4. 執行模型訓練。
5. 查看資料摘要：
 - 原始資料筆數
 - 清理後資料筆數
 - train / validation / test 筆數
6. 查看模型比較表：
 - Accuracy

- F1
 - Strategy return
 - Buy & Hold return
 - Sharpe ratio
7. 查看最佳模型與 selected threshold ◦
 8. 查看權益曲線比較：
 - 策略權益曲線
 - Buy & Hold 權益曲線
 9. 查看最新交易訊號：
 - P(up)
 - target position
 10. 下載 backtest CSV ◦

Demo 核心訊息

此 dashboard 展示完整的資料科學流程：

原始 OHLCV 股價資料

→ 技術指標特徵工程

→ 監督式模型訓練

→ validation 選模型與 threshold

→ test set 回測

→ 與 Buy & Hold 比較投資績效

專案限制

- 預測目標是隔日漲跌，金融市場雜訊高。
- 目前只使用技術指標，未納入基本面或籌碼面。
- 交易成本是假設值，未完整模擬滑價與流動性。
- 回測是研究用途，並非真實下單系統。
- 技術面策略尚未像 Triple Barrier 策略一樣加入完整停利、停損、ATR 動態風控與倉位管理。

未來改進方向

- 加入 walk-forward validation。
- 加入風險調整後的 threshold selection。
- 加入停利、停損與動態倉位控制。
- 加入交易成本敏感度分析。
- 加入更強的 baseline，例如均線交叉策略、隨機訊號策略。

- 加入正式統計檢定，確認策略改善是否顯著。